

Тема 2. Указатели, арифметика указателей.

Задание 1.1

2. Постановка задачи.

Внутри функции `int main(void) { /*... */ }` определите указатель `double **pointer = NULL;`. Инициализируйте этот указатель адресом другого указателя типа `double *`, который указывает, в свою очередь, на переменную `double`. Используйте `pointer` для записи и чтения в эту переменную значения 2. При этом выполните следующее:

- используйте функции типа `*alloc(...)` для выделения оперативной памяти под динамические объекты;
- запишите и выведите число, указанное в крайней правой окружности, на экран, используя указатель `double **pointer = NULL;`;
- используйте функцию `free(...)` для освобождения оперативной памяти, выделенной под динамические объекты.

3. Математическая модель.

Операция присваивания.

4. Список идентификаторов.

Имя	Тип	Смысл
pointer	double	указатель на указатель на переменную

5. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      double **pointer = NULL;
7
8      |*(*(pointer = (double**) malloc(sizeof(double*))) = (double*) malloc(sizeof(double))) = 2;
9
10     printf("pointer points to: %lf", **pointer);
11
12     free(*pointer);
13     free(pointer);
14
15     return 0;
16 }
17
```

6. Результаты.

```
pointer points to: 2.000000
Process returned 0 (0x0)   execution time : 0.084 s
Press any key to continue.
```

Задание 1.2

2. Постановка задачи.

Напишите программу, которая складывает два числа с использованием указателей на эти числа.

3. Математическая модель.

$$\text{sum} = *p + *m$$

4. Список идентификаторов.

Имя	Тип	Смысл
k	double	Входная переменная
n	double	Входная переменная
sum	double	Результат
p	double	Указатель на переменную k
m	double	Указатель на переменную n

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      double k, n, sum;
7      double *p = &k;
8      double *m = &n;
9
10     printf("Enter the first number\n");
11     scanf("%d", &k);
12     printf("Enter the second number\n");
13     scanf("%d", &n);
14
15     sum = *p + *m;
16
17     printf("Sum = %d", sum);
18
19     return 0;
20 }
21

```

6. Результаты.

```

Enter the first number
12
Enter the second number
22
Sum = 34
Process returned 0 (0x0)   execution time : 2.980 s
Press any key to continue.

```

Задание 1.3

2. Постановка задачи.

Напишите программу, которая находит максимальное число из двух чисел, используя указатели на эти числа.

3. Математическая модель.

$*p < *m; *m < *p; *m = *p$

4. Список идентификаторов.

Имя	Тип	Смысл
k	double	Входная переменная
n	double	Входная переменная
m	double	Указатель на переменную k
p	double	Указатель на переменную n

5. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      double k, n;
7      double *p = &k;
8      double *m = &n;
9
10     printf("Enter the first number\n");
11     scanf("%d", &k);
12     printf("Enter the second number\n");
13     scanf("%d", &n);
14
15     if (*p > *m) printf("Max number = %d", *p);
16     else
17     {
18         if (*p < *m) printf("Max number = %d", *m);
19         else printf("%d = %d", *p, *m);
20     }
21
22     return 0;
23 }
24
```

6. Результаты.

```
Enter the first number
33
Enter the second number
100
Max number = 100
Process returned 0 (0x0)   execution time : 2.550 s
Press any key to continue.
```

Задание 1.4

2. Постановка задачи.

Напишите программу, которая создаёт одномерный динамический массив из чисел с плавающей точкой двойной точности, заполняет его значениями с клавиатуры и распечатывает все элементы этого массива, используя арифметику указателей (оператор +), а не обычный оператор доступа к элементу массива - [].

3. Математическая модель.

$ptr + i$, где i – счётчик цикла.

4. Список идентификаторов.

Имя	Тип	Смысл
A	array (со значениями double)	Массив входных значений
i	int	Счётчик цикла
ptr	double	Указатель на массив входных значений

5. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      int n;
7      double* ptr = NULL;
8      printf("Enter array size\n");
9      scanf("%d", &n);
10     double A[n];
11     ptr = A;
12
13     ptr = (double*) malloc(n*sizeof(double));
14
15     for (int i = 0; i < n; i++)
16     {
17         printf("\nA[%d] = ", i);
18         scanf("%lf", (ptr+i));
19     }
20
21     for (int i = 0; i < n; i++)
22     {
23         printf("\n%lf", *(ptr+i));
24     }
25
26     free(ptr);
27
28     return 0;
29 }
```

6. Результаты.

```

Enter array size
5
A[0] = 1.7
A[1] = 77
A[2] = 8.7
A[3] = 6.5
A[4] = 55
1.700000
77.000000
8.700000
6.500000
55.000000
Process returned 0 (0x0)   execution time : 14.959 s
Press any key to continue.

```

Задание 1.5

2. Постановка задачи.

Вывести элементы динамического массива целых чисел в обратном порядке, используя указатель и операцию декремента (--).

3. Математическая модель.

$p + i$, где i – счётчик цикла.

4. Список идентификаторов.

Имя	Тип	Смысл
i	int	Счётчик цикла
p	int	Указатель на массив
A	array of int	Массив входных значений
n	int	Количество элементов массива

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int n;
7      printf("Enter array size\n");
8      scanf("%d", &n);
9
10     int i;
11     int A[n];
12     int* p = A;
13     p = (int*) malloc(n*sizeof(int));
14
15     for (int i = 0; i < n; i++)
16     {
17         printf("\nA[%d] = ", i);
18         scanf("%d", (p+i));
19     }
20     p = p + (n-1);
21
22     for (int i = 0; i < n; i++)
23     {
24         printf("\n%d", *p);
25         p--;
26     }
27     free(p + 1);
28     return 0;
29 }

```

6. Результаты.

```

Enter array size
5
A[0] = 1
A[1] = 2
A[2] = 3
A[3] = 4
A[4] = 5
5
4
3
2
1
Process returned 0 (0x0)   execution time : 3.992 s
Press any key to continue.

```

Задание 1.6

2. Постановка задачи.

Определите переменную целого типа `int a = 1234567890`; и выведите побайтово её содержимое на экран, используя указатель `char *`.

3. Математическая модель.

`c + i`, где `i` – счётчик цикла.

4. Список идентификаторов.

Имя	Тип	Смысл
a	int	Входная переменная
ptr	int	Указатель на переменную a
c	unsigned char	Указатель на указатель на переменную a
i	int	Счётчик цикла

5. Код программы.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int a = 1234567890;
7      int *ptr;
8      unsigned char *c;
9
10     ptr = &a;
11     c = (unsigned char*) ptr;
12
13     for (int i = 3; i > -1; i--)
14         printf("%0.2x", *(c+i));
15
16     return 0;
17 }
18
```

6. Результаты.

```
499602d2
Process returned 0 (0x0)   execution time : 0.223 s
Press any key to continue.
```

Задание 1.7

2. Постановка задачи.

Выделите память под двумерный динамический массив, используя

массив указателей на строки (см. лекции). Затем освободите корректно оперативную память.

3. Математическая модель.

$A[i] = A_data + i * N$, где $A[i]$ – указатель на элемент матрицы, i – счётчик цикла, N – кол-во столбцов.

4. Список идентификаторов.

Имя	Тип	Смысл
M	int	Количество строк в матрице
N	int	Количество столбцов в матрице
ptr	void	Указатель на память, выделенную для матрицы
A	double	Указатель на матрицу
A_data	double	Указатель на отдельную строку матрицы
i	int	Счётчик цикла
j	int	Счётчик цикла

5. Код программы.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  double** allocate_matrix(const int M, const int N) {
5      void *ptr = malloc (M * sizeof (double *) + M * N * sizeof (double));
6      double **A = (double **) ptr;
7      double *A_data = (double *) (A + M);
8
9      for (int i = 0; i < M; i++) {
10         A[i] = A_data + i*N;
11     }
12
13     return A;
14 }
15
16 void print_matrix(double** A, const int M, const int N) {
17     for(int i = 0; i < M; ++i) {
18         for(int j = 0; j < N - 1; ++j) {
19             printf("%lf, ", A[i][j]);
20         }
21         printf("%lf\n", A[i][N - 1]);
22     }
23 }
24
25 void free_matrix (double** A) {
26     free(A);
27 }
28
29 int main(void) {
30     double **A = allocate_matrix(3, 3);
31
32     A[0][0]=1.5;
33     A[1][0]=2.5;
34     A[2][0]=3.5;
35     A[0][1]=4.5;
36     A[1][1]=5.5;
37     A[2][1]=6.5;
38     A[0][2]=7.5;
39     A[1][2]=8.5;
40     A[2][2]=9.5;
41
42     print_matrix(A, 3, 3);
43
44     free_matrix(A);
45
46     if (A == NULL) return 0;
47 }
48

```

6. Результат выполненной программы.

```
1.500000, 4.500000, 7.500000  
2.500000, 5.500000, 8.500000  
3.500000, 6.500000, 9.500000
```

```
Process returned 0 (0x0)   execution time : 0.160 s  
Press any key to continue.
```